OnionPIRv2: Efficient Single-Server PIR

Yue Chen
yuec12@illinois.edu

Ling Ren
renling@illinois.edu

Siebel School of Computing and Data Science
University of Illinois at Urbana-Champaign

Abstract

This paper presents OnionPIRv2, an efficient implementation of OnionPIR incorporating standard orthogonal techniques and engineering improvements. OnionPIR is a single-server PIR scheme that improves response size and computation cost by utilizing recent advances in somewhat homomorphic encryption (SHE) and carefully composing two lattice-based SHE schemes to control the noise growth of SHE. OnionPIRv2 achieves 3.7× response overhead for 3 KB entries and up to 1372 MB/s server computation throughput in our evaluation.

1 Introduction

Protecting user privacy is becoming a critical concern for cloud applications and service providers. *Private information retrieval* (PIR) is a fundamental cryptographic primitive that enables a user to retrieve an entry from a public database without revealing which entry is retrieved. In this paper, we focus on single-server PIR.

In PIR, there are three well-established efficiency metrics: *request size* (data uploaded from the client to the server), *response size* (data downloaded from the server to the client), and *server computation*. Because server computation is typically roughly proportional to the database size, we follow recent work [24] and measure the server computation cost using *throughput*, defined as the plaintext database size in bytes divided by the server computation time. Another metric pointed out in recent work [24] is *extra* server storage. Many recent PIR schemes require the server to store helper values (key materials) per client. [1]

SealPIR [3] is a milestone in single-server PIR. It has a reasonably small request size, but is still very expensive in response size and server computation. Its response overhead is at least 100 times larger than the plaintext entry being retrieved. Its server computation throughput is around 150 MB/s; that is, it needs about 20 seconds of server computation for a 3 GB database. Park and Tibouchi [28] and Ali et al. [2] presented techniques to reduce response overhead but had to worsen server computation even more.

OnionPIR. In 2021, Mughees et al. designed OnionPIR [26] to address the response bottleneck of single-server PIR, while keeping the server computation cost similar to that of SealPIR. The main technique is to control the noise growth by replacing regular RLWE ciphertext multiplication with a special homomorphic multiplication called *external product* [11], which composes two homomorphic encryption schemes for much superior noise control. However, directly applying external

[1] There are also recent works that require the client to store a large amount of hints or perform heavy computation. We do not use those techniques and do not compare with those schemes in this article.

products will not lead to a practical scheme. Several supporting techniques were invented, most notably a new query-packing algorithm and non-uniform database dimensions.

The paradigm of OnionPIR has since been adopted in follow-up works such as Spiral [24], WhisPIR [14], Respire [9], and Faster Spiral [23]. These subsequent works reported improved performance over the original OnionPIR prototype through a combination of algorithmic changes and engineering refinements. However, the original OnionPIR paper focused on presenting algorithmic ideas, and its open-source prototype had poor performance. This makes it difficult to evaluate the algorithmic changes proposed in subsequent works.

This article. This article presents an improved implementation of OnionPIR incorporating mostly standard orthogonal techniques from the FHE literature and our best engineering efforts. We call the implementation OnionPIRv2. OnionPIRv2 supports multiple parameter choices to offer different trade-offs between communication and server computation. A representative parameter setting achieves 3.7× response overhead and 1031–1372 MB/s server computation throughput, which are up to 13% reduction in response overhead and 11.3× improvement in throughput, respectively, compared to the original OnionPIR prototype.

OnionPIRv2 offers balanced efficiency across all efficiency metrics. In comparison, other works in the OnionPIR paradigm tend to prioritize particular metrics while substantially sacrificing others (see Table 2).

For readers' convenience, we repeat the necessary background and OnionPIR protocol details. The pseudocode in this article reflects additional techniques and has improved clarity.

2 Background and Preliminary

2.1 Somewhat Homomorphic Encryption

State-of-the-art single-server PIR schemes use lattice-based leveled Fully Homomorphic Encryption, also called *Somewhat Homomorphic Encryption* (SHE), whose security relies on the hardness of Learning With Errors (LWE) or its variant on the polynomial ring (RLWE). We will compose two SHE schemes: BFV [6, 15] and RGSW [17, 12]. As its name suggests, a SHE scheme supports a limited number of homomorphic addition and multiplication operations on the ciphertexts. All known SHE schemes use *noisy* ciphertexts. Homomorphic operations on the ciphertexts increase the noise level in the resulting ciphertext. After a certain number of operations, the noise would become too large, and the ciphertext could no longer be decrypted.

BFV encryption. The BFV scheme is defined over a fixed polynomial ring $R = \mathbb{Z}[x]/(x^n + 1)$. Here, n is the degree of the polynomial and is usually a power of two. The secret key s is a polynomial sampled from a distribution of "small" (e.g., with ternary coefficients) polynomials in R. Let q and t denote the coefficient modulus for the ciphertext and plaintext, respectively. A plaintext message m is a polynomial in $R \mod t$. Each ciphertext consists of two polynomials in $R \mod q$ and is given as $(c_0, c_1) = (-as + e + \Delta m, a)$. Here, a is sampled uniformly at random from $R \mod q$, m is the plaintext polynomial, e is a noise polynomial with coefficients sampled from a bounded Gaussian distribution, and $\Delta = \lfloor q/t \rfloor$ is a scaling factor to put the message at the most significant bits. When $\|e\|_\infty < \Delta/2$, a ciphertext can be decrypted by computing

$$\left\lfloor \frac{c_0 + c_1 s}{\Delta} \right\rceil \mod t.$$

Table 1: Comparison of computation costs and noise growths of homomorphic operations. $\ell = 5$ is a typical choice.

Operation	Cost	Noise Growth
BFV ciphertext addition	≈ 0	additive
BFV plaintext-ciphertext multiplication	2	multiplicative
BFV ciphertext-ciphertext multiplication	$4 + 2\ell$	multiplicative
External product	4ℓ	additive (for PIR)

RGSW encryption. The RGSW scheme is parameterized by two parameters: the base B and the gadget vector length ℓ. The two parameters give a trade-off between efficiency and noise growth. The RGSW scheme has a gadget matrix:

$$\mathbf{G} = \mathbf{I}_2 \vee g = \begin{bmatrix} g & 0 \\ 0 & g \end{bmatrix} \in R^{(2\ell \times 2)}$$

where the *gadget vector* $g^{(\ell \times 1)} = (B^{\log q/\log B - 1}, B^{\log q/\log B - 2}, \cdots, B^{\log q/\log B - \ell})$.

A RGSW encryption of a plaintext polynomial $m \in R$ is

$$\mathsf{C} = \mathbf{Z} + m \cdot \mathbf{G}$$

where each row of $\mathbf{Z} \in R^{(2\ell \times 2)}$ is a BFV encryption of 0. Notice that the message m is not multiplied by the scaling factor Δ.

2.2 Noise Growth and Computational Cost of Homomorphic Operations

As mentioned, each homomorphic operation in SHE increases the noise in the output ciphertext. Different operations result in drastically different computation costs (measured by the number of polynomial multiplications) and noise growths. These significantly impact design decisions in PIR. We discuss below the approximate noise growth and computation costs of different operations and summarize them in Table 1.

BFV **ciphertext addition.** This operation adds two BFV ciphertexts and outputs a BFV ciphertext encrypting the plaintext sum. This operation is very efficient (does not involve polynomial multiplication) and has very small noise growth.

BFV **plaintext-ciphertext multiplication.** This operations takes as input a plaintext polynomial m and a BFV ciphertext encrypting m'. The output is a BFV ciphertext encrypting the product $m \cdot m'$. The noise term is multiplied by the plaintext [15]. This operation requires two polynomial multiplications.

BFV **ciphertext-ciphertext multiplication.** This operations takes as input two BFV ciphertexts and outputs a BFV ciphertext encrypting the plaintext product. This operation increases the noise by t (the plaintext modulus). This operation also requires an expensive relinearization step that takes $4 + 2\ell$ polynomial multiplications, where the value of ℓ is similar to the decomposition factor ℓ in RGSW.

It is important to note that BFV multiplications (of either type) make the noise multiply and hence blow up exponentially if performed repeatedly. This is the primary source of inefficiency in prior PIR schemes.

External product. The external product operation takes as input a BFV ciphertext d encrypting m_d and a RGSW ciphertext C encrypting m_C, with respect to the same secret key s. The output

is a BFV ciphertext encrypting the plaintext product $m_d \cdot m_C$. The noise (measured by variance) of the resulting ciphertext is roughly $O\left(nB^2 \cdot \mathsf{Err}(\mathsf{C}) + |m_{\mathsf{C}}| \cdot \mathsf{Err}(\mathsf{d})\right)$ where $\mathsf{Err}(\cdot)$ denotes the noise term in a ciphertext [11]. In the context of PIR, m_{C} is usually a single bit, making the noise $O\left(nB^2 \cdot \mathsf{Err}(\mathsf{C}) + \mathsf{Err}(\mathsf{d})\right)$, which is an *additive* growth.

2.3 Background on Single-Server PIR Protocols

The most basic SHE-based single-server PIR scheme is a simple dot product between the plaintext database and a one-hot query vector of ciphertexts sent by the client. The ciphertext corresponding to the target entry encrypts 1 and all the others encrypt 0. This basic scheme has a request size that is linear in the number of database entries. To reduce the request size, the database is represented as a multi-dimensional hypercube [13, 31]. To access a database entry, the client now sends d query vectors, each consisting of $\sqrt[d]{N}$ ciphertexts, where d is the number of dimensions.

SealPIR [3] proposes a technique called *query compression*, which allows the client to pack many encrypted bits within a single (BFV) ciphertext. The server can unpack this ciphertext into an array of ciphertexts, each encrypting a single bit. The unpacking process requires some client-specific key material, leading to a per-client extra storage on the server. With the help of another standard technique of pseudorandom seed [11] (cf. Section 3.3), SealPIR would achieve a request size of 32 KB for databases up to a few Gigabytes.

However, as mentioned earlier, SealPIR still suffers from large response overhead (100x) and server computation (150 MB/s). Both issues are related to the large noise increase from homomorphic multiplications. Anticipating a large noise increase, SealPIR had to use large ciphertexts to encrypt small plaintexts. This clearly hurts the response overhead. It also hurts server computation because each homomorphic operation performs very little actual work on the underlying plaintext.

3 The OnionPIRv2 Protocol

3.1 A Warm-up Protocol

We first describe a warm-up protocol that reduces the response size at the expense of higher computation. The idea is to use RGSW ciphertexts for client query vectors and use *external product* to evaluate the dot product in every dimension. Thanks to the additive noise growth of the external products in PIR, we can now afford to use more dimensions. In fact, one can set each dimension to 2 and use $\log n$ dimensions. In slightly more detail, the database with N entries is regarded as a $2 \times 2 \times \ldots \times 2$ hypercube of $\log N$ dimensions. The client sends a RGSW ciphertext that encrypts a selection bit for each dimension. The server uses external products to recursively and homomorphically select the correct half of the database at each dimension. This is essentially the protocol proposed in [28, 16].

The warm-up protocol significantly reduces the response size over SealPIR due to the smaller noise growth of external products. Unfortunately, the warm-up protocol would incur much higher server computation than SealPIR. This is because the external product is more computationally intensive than plaintext-ciphertext multiplication (cf. Section 2) by roughly an order of magnitude.

3.2 OnionPIR: Optimizing the Computation

A key idea in OnionPIR is a simple trick to keep the server computation low: stay with BFV plaintext-ciphertext multiplication in the initial dimension and make the initial dimension larger than the remaining dimensions. Let N_i denote the size of the i-th dimension. We set the initial

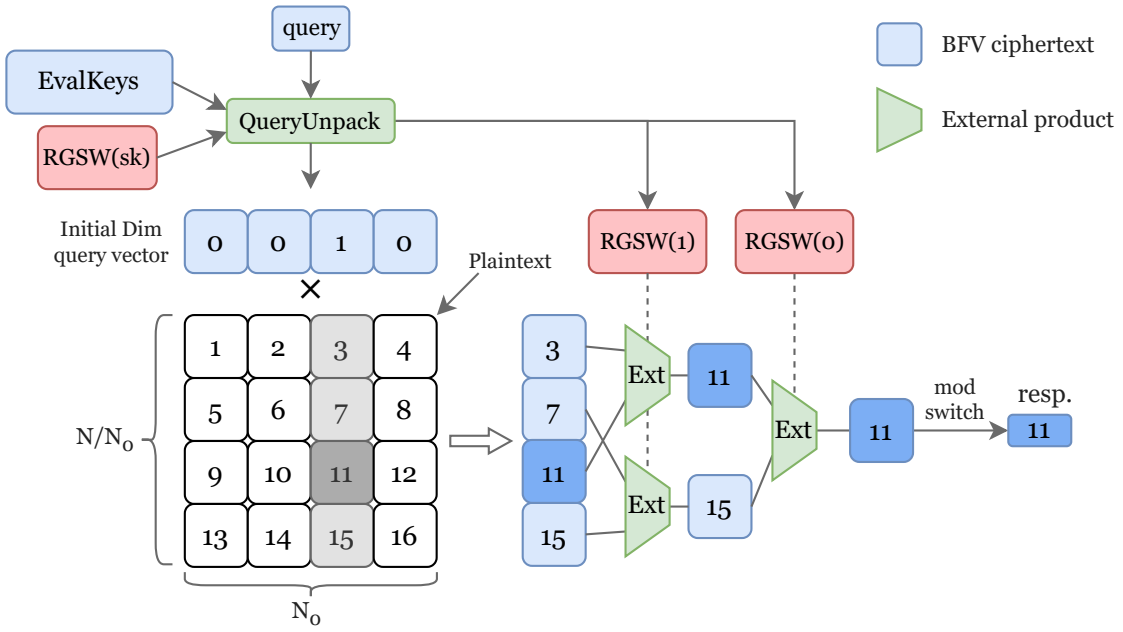

Figure 1: An illustration of the OnionPIRv2 protocol. The QueryUnpack algorithm uses key materials stored on the server. The initial dimension uses plaintext-ciphertext multiplication. The remaining dimensions use external products. The initial dimension is chosen larger, so that it dominates the computation cost.

dimension size N_0 to a few hundred and subsequent dimension sizes to $N_1 = N_2 = \ldots = 2$. This way, the total server computation cost is again dominated by the initial dimension, which uses BFV plaintext-ciphertext multiplications as in prior work. To elaborate, the total number of polynomial multiplications across all dimensions is about $2N + 4\ell \cdot (\frac{N}{2N_0} + \frac{N}{4N_0} + \frac{N}{8N_0} + \cdots)$. With a relatively large N_0 (a few hundred), the $2N$ polynomial multiplications in the initial dimension dominate the computational cost.

Modulus choice. The choice of the ciphertext moduli has a big impact on computation. The total noise growth dictates $\log q - \log t$. Thus, a large ciphertext q decreases the ciphertext expansion factor (i.e., $\log q / \log t$) and improves the server computation throughput. However, a larger q increases the request size and the key material size. Due to this trade-off, we support multiple parameter choices in OnionPIRv2 (though we report results for only one choice in this article).

3.3 Query Compression

Sending one ciphertext per query bit leads to a large request size. SealPIR introduced the query compression technique [3] to reduce the query size. The high-level idea is that the client can pack many *independent* bits into a single ciphertext. The server then unpacks this ciphertext into individual ciphertexts, each encrypting a single bit. SealPIR's query compression is for BFV ciphertexts. We adapt the query compression algorithm for RGSW ciphertexts from [10].

Query packing. Algorithm 1 is the query packing algorithm in OnionPIR. For the initial dimension, we pack N_0 values where N_0 is the size of the initial dimension. For each subsequent dimension, we pack ℓ values, corresponding to the first ℓ rows of the single RGSW ciphertext for

Algorithm 1: QueryPack algorithm of OnionPIR

Input:

- $m_0 \in \{0, \ldots, N_0 - 1\}$, plaintext query index for initial dimension.
- $\{m_i\}_{i=1}^{d-1}$, plaintext query bits, one for each higher dimension.

Output: $\tilde{c}$, a single BFV ciphertext packing the query.

Notation:

- N_0, size of the initial dimension.
- d, the number of dimensions.
- ℓ, the RGSW decomposition factor.
- w, smallest power of two no less than $N_0 + \ell(d-1)$.
- $1/w$, the inverse of w modulo t.
- $g = (B^{\log q/\log B - 1}, \cdots, B^{\log q/\log B - \ell})$, the RGSW gadget.
- $[\cdot]$, accessing an element of a vector.
- $\langle\cdot\rangle$, accessing a coefficient of a polynomial.
- $\mathsf{BitRev}(x)$, bit reversal of integer x in $(\log w)$-bit representation.

Create a zero plaintext pt
$\mathsf{pt}\langle\mathsf{BitRev}(m_0)\rangle = 1/w$
$\tilde{c} = (\tilde{c}_0, \tilde{c}_1) = \mathsf{BFV}(\mathsf{pt})$
$g' = g/w$
for $i = 1 : d-1$ **do**
  if $m_i == 1$ **then**
    for $k = 0 : \ell - 1$ **do**
      $p = \mathsf{BitRev}(N_0 + \ell(i-1) + k)$
      $\tilde{c}_0\langle p\rangle = (\tilde{c}_0\langle p\rangle + g'[k]) \mod q$
    end
  end
end
Outputs $\tilde{c}$.

that dimension (see Section 3.4). With all subsequent dimensions having size 2, the number of dimensions $d = 1 + \lceil \log_2(N/N_0) \rceil$. This gives a total of $N_0 + \ell(d-1)$ values to pack. A BFV ciphertext in our implementation has at least $n = 2048$ plaintext slots, so we can pack all these values in a single BFV ciphertext.

Query unpacking. Algorithm 3 unpacks a single BFV query into individual ciphertexts. It first calls Algorithm 2 to extract the coefficients of the underlying polynomial of the packed query. A "substitution" operation $\mathsf{Subs}(c, j)$ maps $c = \mathsf{BFV}(\sum m_i x^i)$ to $\mathsf{BFV}(\sum m_i x^{ij})$ for any odd j. The resulting ciphertext is encrypted under a new secret key $s(x^j)$, but we can use the key switching technique [7] to switch the secret key back to $s(x)$. Setting $j = n + 1$ to exploits the relation $x^{i(n+1)} = (x^n)^i \cdot x^i = (-1)^i \cdot x^i$ in the ring. We can then use

$$c + \mathsf{Subs}(c, n+1) = \mathsf{BFV}(\sum 2m_{2i} \cdot x^{2i})$$
$$c + x^{-1}(c - \mathsf{Subs}(c, n+1)) = \mathsf{BFV}(\sum 2m_{2i+1} \cdot x^{2i})$$

Algorithm 2: $\mathsf{ExpandBFV}$ algorithm adapted from Algorithm 3 in [10].

Input: $\tilde{c}$ = Output of Algorithm 1

Output: $\mathsf{c}[i]$, $0 \leq i < u$, an array of BFV ciphertexts, each encrypting a constant polynomial corresponding to a value packed by Algorithm 1.

Notation: $u = N_0 + \ell(d-1)$, number of packed values. See Algorithm 1 for other notation.

$\mathsf{c}[1] = \tilde{c}$
for $i = 1$ **to** $w - 1$ **do**
  $k = 2^{\lfloor \log i \rfloor}$
  if $iw/k - w < u$ **then**
    $c' = \mathsf{Subs}(\mathsf{c}[i],\, w/k + 1)$
    $\mathsf{c}[2i] = \mathsf{c}[i] + c'$
    $\mathsf{c}[2i+1] = (\mathsf{c}[i] - c')\, x^{-k}$
  end
end
Outputs $\mathsf{c}[w, \ldots, w + u - 1]$

Algorithm 3: $\mathsf{QueryUnpack}$ algorithm, adapted from Algorithm 4 in [10].

Input: $(\tilde{c})$, a single BFV ciphertext packing the query; $A = \mathsf{RGSW}(s)$, RGSW encryption of the client's secret key s, provided by client during initialization.

Output: $\left(\mathsf{C_{BFV}}, \{\mathsf{C_{RGSW}}[i]\}_{i=1}^{d-1}\right)$, unpacked encrypted query vectors. $\mathsf{C_{BFV}}$ is the vector of N_0 BFV ciphertexts for the initial dimension and $\mathsf{C_{RGSW}}[i]$ is the single RGSW ciphertext for the i-th dimension.

Notation: $\mathsf{C_{RGSW}}[i]_k$ denotes the k^{th} row of the $\mathsf{C_{RGSW}}[i]$ ciphertext.

$\mathsf{c} = \mathsf{ExpandBFV}(\tilde{c})$
$\mathsf{C_{BFV}}[0, \ldots, N_0 - 1] = \mathsf{c}[0 \ldots, N_0 - 1]$
for $i = 1 : d - 1$ **do**
  $base = N_0 + \ell(i-1)$
  for $k = 0 : \ell - 1$ **do**
    $\mathsf{C_{RGSW}}[i]_k = \mathsf{c}[base + k]$
    $\mathsf{C_{RGSW}}[i]_{k+\ell} = \mathsf{ExternalProduct}(A, \mathsf{c}[base + k])$
  end
end
Outputs $\left(\mathsf{C_{BFV}}, \{\mathsf{C_{RGSW}}[i]\}_{i=1}^{d-1}\right)$.

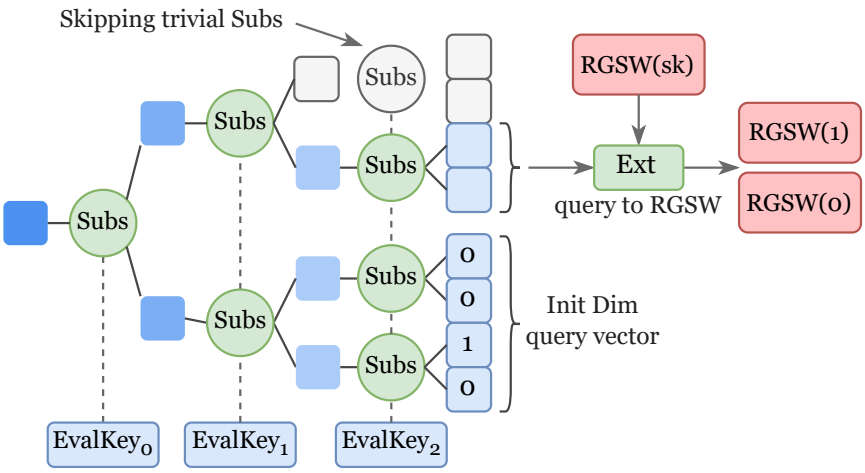

Figure 2: An illustration of Algorithm 3. One evaluation key is used for Subs in each level.

to extract the coefficients at even and odd positions, respectively. By recursively extracting even and odd coefficients at each level, Algorithm 2 extracts all coefficients from the packed query into individual ciphertexts, each encrypting a constant.

At this point, the first N_0 ciphertexts are used as the encrypted query vector for the initial dimension. For RGSW selection ciphertexts for subsequent dimensions, Algorithm 2 outputs only the top ℓ rows of each RGSW ciphertext (because Algorithm 1 packed only those rows). Algorithm 3 then derives the bottom ℓ rows by performing external products between the top rows and an RGSW encryption of the client secret key; see Section 4.3 of [10] for a more detailed explanation.

More details in query packing. At a high level, query packing (Algorithm 1) adds different values to distinct coefficients of a BFV(0) ciphertext, and ExpandBFV (Algorithm 2) shifts each packed value to the constant term of its corresponding ciphertext. Concretely, packing a value is done by inserting the monomial μx^i to the first polynomial of BFV(0), producing $c = (\mu x^i, 0) + (-as+e, a)$. After running algorithm 2 on c, the resulting ciphertext at position BitRev(i) is exactly $(-a's + e' + 2^h\mu, a')$, where h is the height of the expansion tree, a' and e' are the new random polynomial and noise term, respectively.

Pseudorandom component in BFV. Recall that a BFV ciphertext consists of two components, and one of them is sampled uniformly randomly from $R \mod q$. So, the client can generate it pseudorandomly from a short random seed and send the seed to the server. This trick is proposed in [11] and cuts the request size in half.

3.4 Standard Techniques Incorporated by OnionPIRv2

OnionPIRv2 incorporates many techniques overlooked in the original OnionPIR. Some are known or orthogonal techniques in the FHE literature (this subsection), while others are new observations (next subsection).

Modulus switching. *Modulus switching* [7, 2] is a standard technique to reduce the size of a RLWE ciphertext. It takes a ciphertext with modulus q and scales it to a ciphertext with a smaller modulus $q' < q$, while maintaining correct decryption. This is possible because the noise norm of

the ciphertext after modulus switching is also rescaled to approximately q'/q of the original noise. Modulus switching is often applied when no further computation needs to occur on the ciphertext. For our context, it is applied to the final response to the client.

Modulus switching also supersedes a design in the original OnionPIR: the plaintext splitting in the initial dimension was aimed to reduce the response size, but it is less effective than modulus switching and increases computation.

Composite modulus and composite NTT. In OnionPIR, the ciphertext modulus q has nearly 64 bits (or more), so the algorithm requires many $64 \times 64 \to 128$-bit integer multiplications. One approach is to use a composite modulus $q = q_1 q_2$ together with the Chinese Remainder Theorem (CRT), so we can instead rely on the faster $32 \times 32 \to 64$-bit integer multiplications. However, naïvely doing so would force all the entire PIR algorithm to operate under CRT. Although CRT speeds up the initial dimension, it hurts subsequent dimensions because gadget decomposition would require repeated CRT and inverse CRT conversions. The composite Number Theoretic Transform (NTT) technique introduced by [21, 23] helps resolve this tension: it enables CRT-based $32 \times 32 \to 64$-bit integer multiplications in the initial dimension, while allowing subsequent dimensions to treat the composite q as a single modulus to avoid repeated CRT conversions.

BV-style key switching. The original OnionPIRis built on the SEAL library [30]. The key switching operation in SEAL uses a "hybrid" method [20] that temporarily increases the modulus size during key switching, which either reduces the security level or requires less efficient parameters. For better security and efficiency, OnionPIRv2 uses the BV-style key switching [8]. We refer readers to [20] for discussions on the trade-offs of different key switching techniques.

Signed gadget decomposition and two sets of decomposition parameters. The original OnionPIR uses an unsigned gadget decomposition, meaning that the decomposed values are within $[0, B)$, where B is the decomposition base. Signed decomposition [25, 24], where the decomposed values fall within $[-B/2, B/2)$, helps reduce the noise growth of external products because the decomposed values have smaller L_∞ norms.

When using external products, a larger ℓ (which implies a small B) incurs higher computation but has smaller noise growth. External products are used in two places in OnionPIR. In query unpacking (Algorithm 3), only a logarithmic number of external products are used, but the noise carries over to the rest of the protocol. In the remaining dimensions (Algorithm 4), external products are used many times. Because of their different trade-offs, Spiral [24] suggests using different parameters for the two places: a larger ℓ in query unpacking and a smaller ℓ in remaining dimensions. OnionPIRv2 adopts this trick.

One external product per selection. Starting from the second dimension, we select one half of the database in each dimension. Let us think of the database as two halves x and y, and denote the choice bit by b. OnionPIR uses the naïve method computes $(1-b) \cdot x + b \cdot y$. A better method is to compute $b \cdot (y - x) + x$ instead. This simple and standard trick uses only one external product for each 1-out-of-2 selection in higher dimensions [11].

Delayed modular reduction and Barrett reduction. By default, homomorphic operations in mainstream FHE libraries apply the modular reduction after each homomorphic addition or multiplication. However, since we need to perform many homomorphic additions sequentially, it is

more efficient to skip the modular reductions in between and only perform the modular reduction once at the very end.

In modular arithmetic, a naïve way to compute $r = x \mod n$ is using $r = x - \lfloor x/n \rfloor \cdot n$. However, divisions are slow in practice. If we know the modulus n ahead of time, with some pre-computation, Barrett reduction replaces divisions with cheap bit-shifting operations. We refer readers to [4, 18] for the original ideas and detailed implementations.

3.5 New Optimizations in OnionPIRv2

Tree pruning in query unpacking. Conceptually, the query unpacking process in Algorithm 2 has a tree structure: each Subs call splits a node into two children, until reaching the leaf nodes that correspond to the $N_0 + \ell(d-1)$ BFV ciphertexts. In the original OnionPIR, some leaf nodes are unused zero ciphertexts, and the intermediate nodes leading to them are also guaranteed to be zero. The Subs calls on these zero ciphertexts are wasteful. OnionPIRv2 avoids this waste in two ways. First, our implementation supports any choice of N_0 so that we can try to set $N_0 + \ell(d-1)$ to a power of 2. Second, if the optimal performance calls for different parameterization, we skip those unnecessary Subs calls on zero ciphertexts by pruning those nodes (line 4 of Algorithm 2). To do so, we need to pack the values in a bit-reversed fashion (hence the use of $\mathsf{BitRev}(\cdot)$).

Pseudorandom components in key materials The pseudorandom component idea can also reduce the key material size by half. The trick directly applies to the key-switching keys used in the Subs automorphism in Algorithm 2. The $\mathsf{RGSW}(s)$ in Algorithm 3 requires extra care.

Recall that $\mathsf{RGSW}(s) = \mathbf{Z} + s \cdot \mathbf{G}$, where s is the encryption secret key itself, $\mathbf{Z}$ is 2ℓ rows of $\mathsf{BFV}(0)$, and $\mathbf{G} = \mathbf{I}_2 \vee g$ is the gadget matrix. Let g_k denote the k^{th} gadget values, $k \in \{0, \ldots, \ell-1\}$. We first observe that the top ℓ rows of $\mathsf{RGSW}(s)$ have the form $(-a \cdot s + e + s \cdot g_k, a)$. The second term can be easily generated pseudorandomly using a seed, so the server does not have to store it. The bottom ℓ rows of $\mathsf{RGSW}(s)$ have the form $c = (-a \cdot s + e, a + s \cdot g_k)$. We can equivalently write it as $c' = (-(a - s \cdot g_k) \cdot s + e, a)$. The correctness can be verified by observing that c and c' decrypt to the same result, namely, $c_0 + c_1 \cdot s = c'_0 + c'_1 \cdot s$. Another way to interpret the equivalence is to think of $a - s \cdot g_k$ as a'. Since the polynomial a in each row is (pseudo-)random, applying a shift $-s \cdot g_k$ to a does not change its distribution. We can now use a single short seed to generate the second components of all 2ℓ rows of $\mathsf{RGSW}(s)$. This cuts the server storage for $\mathsf{RGSW}(s)$ in half.

Using standard matrix multiplication. The initial dimension computation can be viewed as a generalized matrix multiplication between the database, represented as a matrix of size $(N/N_0) \times N_0$, and the query BFV vector of size $N_0 \times 2$, where each element is a polynomial of degree n.

The polynomial multiplications are performed using Number Theoretic Transformation (NTT). After applying NTT to the plaintext database, each polynomial multiplication becomes an element-wise vector multiplication. Then, we can reinterpret the computation as n separate instances of standard matrix multiplication, each corresponding to one component after NTT. This way, each instance (which we call a *slice*) becomes a standard integer matrix multiplication between an integer matrix of size $(N/N_0) \times N_0$ and an integer matrix of size $N_0 \times 2$. With our choice of N_0 (a few hundred), the latter matrix is small enough to fit in cache, so the runtime of each slice is dominated by memory access time for the larger matrix. Ideally, the computation of the initial dimension can approach the bandwidth limit of reading the NTT-preprocessed database from the main memory.

Algorithm 4: OnionPIR Protocol.

Input: DB server database of size N; idx, the index of the client's desired entry.

Notation: All notations defined in Algorithm 1 and 3, and
- N, database size.
- $\mathsf{DB}[i]$, i-th entry.
- DB', intermediate database.
- shaded part is executed by server.

Client converts idx into a vector $(i_1, \cdots i_d)$, where i_j is the position of idx entry in j-th dimension of the hypercube.

Client computes $\tilde{c} = \mathsf{QueryPack}\left((i_1, \cdots i_d)\right)$, and sends $\tilde{c}$ to the server.

Server computes $(\mathsf{C_{BFV}}, \{\mathsf{C_{RGSW}}[i]\}_{i=1}^{d-1}) = \mathsf{QueryUnpack}(\tilde{c})$

for $j = 0 : N/N_0 - 1$ **do**

  $\mathsf{DB}'_j = \sum_{k=0}^{N_0-1} \mathsf{C_{BFV}}[k] \cdot \mathsf{DB}[k(N/N_0) + j]$

end

for $i = 1 : d - 1$ **do**

  $half = |\mathsf{DB}'|/2$

  for $j = 0 : half - 1$ **do**

    $\mathsf{DBdiff} = \mathsf{DB}'[half + j] - \mathsf{DB}'[j]$

    $\mathsf{DB}'[j] = \mathsf{ExternalProduct}(\mathsf{C_{RGSW}}[i], \mathsf{DBdiff}) + \mathsf{DB}'[j]$

  end

  $\mathsf{DB}' = \mathsf{DB}'[0, \ldots, half - 1]$

end

Server computes response $r = \mathsf{ModSwitch}(\mathsf{DB}')$ and sends r to the client.

Client decrypts r to get the content of entry idx.

3.6 OnionPIRv2 Full Protocol

The final OnionPIRv2 protocol is given in Algorithm 4. We have introduced all the components of the algorithm separately in previous sections. Below we describe the protocol putting together all the techniques.

The database is represented as a hypercube of d dimensions. The size of the initial dimension is N_0, and each of the remaining dimensions is of size 2. The total number of dimensions is thus $d = 1 + \lceil \log_2(N/N_0) \rceil$.

The client converts the desired index idx into d query vectors, one for each dimension of the hypercube. The client then packs all the query bits into a single BFV ciphertext using Algorithm 1 and sends the ciphertext to the server. The server unpacks this single ciphertext using Algorithm 3 into a vector of BFV ciphertexts for the initial dimension and a single RGSW ciphertext for each subsequent dimension.

For the initial dimension, the server performs a dot product between the BFV ciphertext vector and the plaintext database hypercube. The output is a BFV ciphertext hypercube of one fewer dimension. The server then continues to process higher dimensions similarly but uses external products. After the dot product at each dimension, the output is an intermediate hypercube of one fewer dimension and the input to the next dot product. The output after the last dot product is a single BFV ciphertext encrypting the desired entry. The server applies modulus switching to this ciphertext and sends it to the client. The client decrypts it to obtain the desired database entry.

4 Implementation and Evaluation

4.1 Parameter Choices

The OnionPIR algorithm can use a wide range of RLWE parameter choices. We provide one example set of parameter below:

- Polynomial degree $n = 2048$, 58-bit ciphertext modulus q, 13-bit plaintext modulus t, 22-bit ciphertext modulus q' after modulus switching;
- The gadget decomposition parameter is $\ell = 8$ for key-switching, $\ell = 10$ for external products in query unpacking, and $\ell = 6$ for external products in remaining dimensions;
- The noise for fresh RLWE and RGSW ciphertexts follows a discrete Gaussian distribution with standard deviation $\sigma_{\text{std}} = 2.55$.[2]

These parameters give an estimated security level of 117 bits using the LWE estimator [1].

4.2 Implementation Details and Evaluation Setup

OnionPIRv2 removes the dependency on the SEAL Homomorphic Encryption Library [30], and directly uses the HEXL library [5] (which uses AVX512 instructions) to accelerate common computations in homomorphic encryption, including NTT and vector operations. Our implementation is available at `https://github.com/chenyue42/OnionPIRv2`.

We test the performance of OnionPIRv2 on an Intel(R) Xeon(R) Platinum 8358 CPU @ 2.60 GHz in a single thread. We use Ubuntu 22.04 and GCC 13.3.0 for all our evaluations.

4.3 Evaluation Results

We compare with OnionPIRv1 [26], SpiralPIR [24], and KsPIR [22] in Table 2. We evaluate all the schemes using two database sizes. We use each scheme's "native" (most preferred) entry size. We report the per-client server storage size, request size, response size (along with the multiplicative response overhead), and server computation throughput.

Communication cost. In OnionPIRv2, the request size is fixed for any practical database, which is $n \log q + |\mathsf{seed}|$ bits long, where $|\mathsf{seed}|$ is the size of a short pseudorandom seed. The size of a response ciphertext is $2n \log q'$, where q' is the smaller modulus after modulus switching. If the database exceeds the plaintext size, multiple response ciphertexts are needed. With the parameter choice in our evaluation, the request size is 15 KB and the response size is 11 KB. With a 3 KB entry size, the response blowup is $11/3 \approx 3.7$.

Key material size. The key material size of OnionPIRv2 is about 1.5 MB in the evaluated setting, significantly smaller than all three baselines. This can be stored on the server. Alternatively, it can be sent as part of the request if a system prefers to avoid per-client extra server storage.

Server Computation. The server mainly performs three tasks: (i) query unpacking, (ii) the initial-dimension dot products between the query vector and the database, and (iii) the multiplexers in the remaining dimensions. The first dimension dominate the server's computation, though the other two are not negligible.

[2] Other works [29, 24] describe the noise distribution as "sub-Gaussian with width parameter σ_{width}". The conversion between the two descriptions is $\sigma_{\text{width}} = \sqrt{2\pi} \cdot \sigma_{\text{std}}$.

	OnionPIRv1†	Spiral	KsPIR	OnionPIRv2
DB Entry Size	30 KB	8 KB	8 KB	3 KB
Server Storage	6.3 MB	15–17 MB	9–9.3 MB	1.5 MB
DB Size		1 GB		
Request Size	64 KB	14 KB	140 KB	15 KB
Response Size	128 KB	20.5 KB	26 KB	11 KB
Resp. Overhead	4.27×	2.56×	3.25×	3.7×
Throughput	110 MB/s	289 MB/s	1349 MB/s	1031 MB/s
DB Size		8 GB		
Request Size	64 KB	14 KB	140 KB	15 KB
Response Size	128 KB	21 KB	26 KB	11 KB
Resp. Overhead	4.27×	2.625×	3.25×	3.7×
Throughput	121 MB/s	333 MB/s	1414 MB/s	1372 MB/s

Table 2: Performance comparison of OnionPIRv1, SpiralPIR, KsPIR, and OnionPIRv2. For each metric, we mark the best scheme in green and schemes close to the best in lighter green. †: The OnionPIR implementation has only 74 bits of security due to the use of hybrid key switching.

As explained in Sections 3.4 and 3.5, the initial dimension in OnionPIRv2 is expressed as standard integer matrix multiplications over 29-bit integers (padded to 32-bit), which can approach the performance limit of a single CPU core. In particular, our $32 \times 32 \to 64$-bit integer matrix multiplication achieves roughly 10 GB/s throughput, while the CPU used in our experiments provides a peak single-thread memory bandwidth of around 12 GB/s. To speed up online server computation, the PIR algorithm preprocesses the database by applying NTT on each plaintext entry. This expands the plaintext database by roughly a factor $\log q / \log t$. After rounding, our parameter choices yield a database expansion factor of about 5×. Accounting for this expansion, the server computation throughput in the initial dimension is approximately 1.8 GB/s. Additional computations from query unpacking and subsequent dimensions further reduce the end-to-end server computation throughput to around 1.3 GB/s.

We remark that the $\log q / \log t$ ratio (after padding) is a deciding factor in the server computation throughput. This ratio is ≈ 8 in Spiral [24], ≈ 5 in OnionPIRv2, and ≈ 3.5 in KsPIR. This partly explains the difference in server computation in the three schemes.

5 Conclusion

In this paper, we described OnionPIRv2, an efficient single-server PIR implementation with a response overhead of 3.7× and over 1.3 GB/s throughput.

With our algorithmic and engineering improvements, the initial dimension is reduced to standard matrix multiplications with small integers whose throughput approaches the single-core memory-bandwidth limit. Further algorithmic improvements within the framework of OnionPIR likely require reducing the ciphertext expansion factor or eliminating the NTT preprocessing. Another promising direction is hardware acceleration, for which recent work has begun to emerge [19].

Acknowledgment. We thank Muhammad Haris Mughees for pointing us to the delayed modulus technique. We thank Zhikun Wang for suggesting the pseudorandom component in key materials. We thank Pranav Arunachalaramanan and Hao Chen for helpful discussions.

This work is funded in part by the National Science Foundation Award #2246386 and the Google Research Scholar Program. This work used AWS through the CloudBank project [27], which is supported by National Science Foundation grant #1925001.

References

[1] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *J. Math. Cryptol.*, 9(3):169–203, 2015.

[2] Asra Ali, Tancrède Lepoint, Sarvar Patel, Mariana Raykova, Phillipp Schoppmann, Karn Seth, and Kevin Yeo. Communication-computation trade-offs in PIR. *IACR Cryptol. ePrint Arch.*, 2019:1483, 2019.

[3] Sebastian Angel, Hao Chen, Kim Laine, and Srinath T. V. Setty. PIR with compressed queries and amortized query processing. In *2018 IEEE Symposium on Security and Privacy*, pages 962–979, San Francisco, California, USA, 2018. IEEE Computer Society.

[4] Paul Barrett. Implementing the rivest shamir and adleman public key encryption algorithm on a standard digital signal processor. In Andrew M. Odlyzko, editor, *Advances in Cryptology — CRYPTO' 86*, pages 311–323, Berlin, Heidelberg, 1987. Springer Berlin Heidelberg.

[5] Fabian Boemer, Sejun Kim, Gelila Seifu, Fillipe D.M. de Souza, and Vinodh Gopal. Intel hexl: Accelerating homomorphic encryption with intel avx512-ifma52. In *Proceedings of the 9th on Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, WAHC '21, page 57–62, New York, NY, USA, 2021. Association for Computing Machinery.

[6] Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical gapsvp. In *Annual cryptology conference*, pages 868–886. Springer, 2012.

[7] Zvika Brakerski and Vinod Vaikuntanathan. Efficient fully homomorphic encryption from (standard) lwe. In *2011 IEEE 52nd Annual Symposium on Foundations of Computer Science*, pages 97–106, 2011.

[8] Zvika Brakerski and Vinod Vaikuntanathan. Fully Homomorphic Encryption from Ring-LWE and Security for Key Dependent Messages. In *Advances in Cryptology – CRYPTO 2011*, pages 505–524. Springer, Berlin, Germany, 2011.

[9] Alexander Burton, Samir Jordan Menon, and David J Wu. Respire: High-rate pir for databases with small records. *Cryptology ePrint Archive*, 2024.

[10] Hao Chen, Ilaria Chillotti, and Ling Ren. Onion ring ORAM: efficient constant bandwidth oblivious RAM from (leveled) TFHE. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security, CCS*, pages 345–360, London, UK, 2019. ACM.

[11] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In *Advances in Cryptology - ASIACRYPT- 22nd International Conference on the Theory and Application of Cryptology and Information Security*, pages 3–33, Hanoi, Vietnam, 2016. eprint.

[12] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: fast fully homomorphic encryption over the torus. *J. Cryptol.*, 33(1):34–91, 2020.

[13] Benny Chor, Eyal Kushilevitz, Oded Goldreich, and Madhu Sudan. Private information retrieval. *J. ACM*, 45(6):965–981, 1998.

[14] Leo de Castro, Kevin Lewi, and Edward Suh. Whispir: Stateless private information retrieval with low communication. *Cryptology ePrint Archive*, 2024.

[15] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. *IACR Cryptol. ePrint Arch.*, 2012:144, 2012.

[16] Craig Gentry and Shai Halevi. Compressible fhe with applications to pir. In Dennis Hofheinz and Alon Rosen, editors, *Theory of Cryptography*, pages 438–464, Cham, 2019. Springer International Publishing.

[17] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In *Advances in Cryptology - CRYPTO 2013 - 33rd Annual Cryptology Conference*, pages 75–92, Santa Barbara, CA, USA, 2013. Springer.

[18] Rémi Géraud, Diana MaimuŢ, and David Naccache. *Double-Speed Barrett Moduli*, pages 148–158. Springer Berlin Heidelberg, Berlin, Heidelberg, 2016.

[19] Hyesung Ji, Hyunah Yu, Jongmin Kim, Wonseok Choi, G. Edward Suh, and Jung Ho Ahn. GPIR: Enabling Practical Private Information Retrieval with GPUs. *arXiv*, April 2026.

[20] Andrey Kim, Yuriy Polyakov, and Vincent Zucca. Revisiting homomorphic encryption schemes for finite fields. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021*, pages 608–639, Cham, 2021. Springer International Publishing.

[21] Zhihao Li, Ying Liu, Xianhui Lu, Ruida Wang, Benqiang Wei, Chunling Chen, and Kunpeng Wang. Faster Bootstrapping via Modulus Raising and Composite NTT. *TCHES*, 2024(1):563–591, 2024.

[22] Ming Luo, Feng-Hao Liu, and Han Wang. Faster fhe-based single-server private information retrieval. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 1405–1419, New York, NY, USA, 2024. Association for Computing Machinery.

[23] Ming Luo and Mingsheng Wang. Faster Spiral: Low-Communication, High-Rate Private Information Retrieval. *Cryptography*, 9(1):13, February 2025.

[24] Samir Jordan Menon and David J Wu. Spiral: Fast, high-rate single-server pir via fhe composition. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 930–947. IEEE, 2022.

[25] Daniele Micciancio and Chris Peikert. Trapdoors for Lattices: Simpler, Tighter, Faster, Smaller. In *Advances in Cryptology – EUROCRYPT 2012*, pages 700–718. Springer, Berlin, Germany, 2012.

[26] Muhammad Haris Mughees, Hao Chen, and Ling Ren. Onionpir: Response efficient single-server pir. In *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security*, pages 2292–2306, 2021.

[27] Michael Norman, Vince Kellen, Shava Smallen, Brian DeMeulle, Shawn Strande, Ed Lazowska, Naomi Alterman, Rob Fatland, Sarah Stone, Amanda Tan, et al. Cloudbank: Managed services to simplify cloud access for computer science research and education. In *Practice and Experience in Advanced Research Computing 2021: Evolution Across All Dimensions*, pages 1–4. 2021.

[28] Jeongeun Park and Mehdi Tibouchi. SHECS-PIR: somewhat homomorphic encryption-based compact and scalable private information retrieval. In *Computer Security - ESORICS 2020 - 25th European Symposium on Research in Computer Security*, pages 86–106, Guildford, UK, 2020. Springer.

[29] Chris Peikert. A Decade of Lattice Cryptography. *Found. Trends. Theor. Comput. Sci.*, 10(4):283–424, March 2016.

[30] Microsoft SEAL (release 4.3). `https://github.com/Microsoft/SEAL`, April 2026. Microsoft Research, Redmond, WA.

[31] Julien P. Stern. A new efficient all-or-nothing disclosure of secrets protocol. In *Advances in Cryptology - ASIACRYPT '98, International Conference on the Theory and Applications of Cryptology and Information Security*, volume 1514 of *Lecture Notes in Computer Science*, pages 357–371, Beijing, China, 1998. Springer.